

INSTITUTO FEDERAL DO RIO GRANDE DO SUL - IFRS
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
João Pedro Vargas - Luiz Mário Marangon - Maiki Scalvi - Vinicius Bessega

SISTEMAS DE GERENCIAMENTO DE FROTAS
– DOCUMENTAÇÃO TÉCNICA –

PROFESSOR: JORGE ARANDA
2026

1. Visão Geral do Sistema	3
1.1 Tecnologias Utilizadas	3
2. Arquitetura do Sistema	3
2.1 Estrutura dos Pacotes do Projeto	4
2.2 Relacionamentos Entre Entidades	5
2.3 Diagrama de Classes	6
2.4 Fluxograma Operacional do Projeto	7
3. Padrões de Projeto	7
3.1 Cadastro de Veículos	7
3.2 Cadastro de Motoristas	7
3.3 Validação de Categoria da CNH x Capacidade do Veículo	8
3.4 Início de Viagem	8
3.5 Finalização da Viagem	9
3.6 Cálculo de Custo	9
3.7 Rotas e Integração GPS	9
4. Padrões de Projeto Utilizados	9
4.1 Factory	9
4.2 Facade	9
4.3 Strategy	10
4.4 Observer	10
4.5 Adapter	10
5. FrontEnd	10
6. Relatórios	10
7. Divisão de Responsabilidades	11
8. Banco de Dados	11

1. Visão Geral do Sistema

O Sistema de Gerenciamento de Frotas é uma aplicação web desenvolvida com Spring Boot (backend) e Thymeleaf juntamente com HTML/Bootstrap (frontend), desenvolvido nas matérias de Frameworks e Programação com Padrões de Projeto. Projetado e desenvolvido para fazer um controle de veículos, motoristas, rotas e viagens em uma frota de transporte, o sistema oferece funcionalidades para cálculo automático de custos, geração de relatórios em PDF e Excel, autenticação de usuários além de integração com APIs de GPS.

1.1 Tecnologias Utilizadas

Camada	Tecnologia	Versão
Backend	Java + Spring Boot	3.2.5
Persistência	Spring Data JPA + PostgreSQL	—
Segurança	Spring Security + JWT (JJWT)	0.11.5
Relatórios	iTextPDF + Apache POI	5.5.13 / 5.2.5
Frontend	HTML5 + Bootstrap	5.3.0
Mapas	Leaflet + OpenStreetMap	1.9.4
GPS / Rotas	OpenRouteService + Nominatim	—
Build	Maven	3.9.x

2. Arquitetura do Sistema

O projeto apresenta uma arquitetura em 3 camadas, dividindo as responsabilidades entre Controller, Service, Repository e Model.

2.1 Estrutura dos Pacotes do Projeto

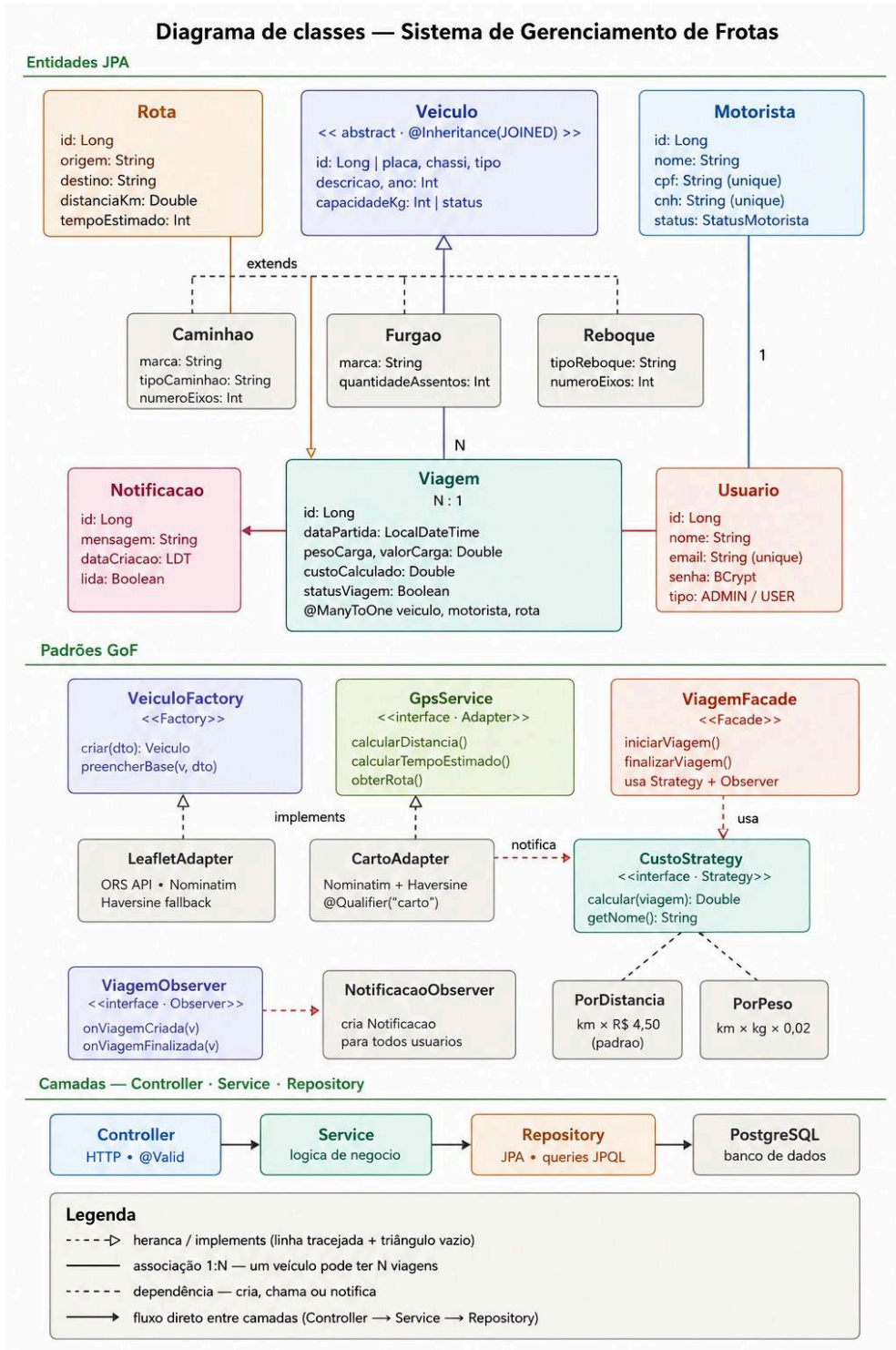
```
gerenciamento.frotas.TrabFinal/  
├── config/  
│   ├── SecurityConfig.java      # Spring Security + CORS  
│   ├── JwtUtil.java             # Geração e validação de tokens  
│   └── JwtAuthFilter.java       # Filtro de autenticação  
├── controller/  
│   ├── AuthController.java      # POST /api/auth/login  
│   ├── VeiculoController.java   # CRUD /api/veiculos  
│   ├── MotoristaController.java # CRUD /api/motoristas  
│   ├── RotaController.java      # CRUD /api/rotas + GPS  
│   ├── ViagemController.java    # /api/viagens  
│   ├── NotificacaoController.java  
│   ├── UsuarioController.java  
│   └── RelatorioController.java # PDF e Excel  
├── dto/  
│   ├── veiculo/                # VeiculoRequestDTO, VeiculoResponseDTO  
│   ├── motorista/              # MotoristaRequestDTO, MotoristaResponseDTO  
│   ├── rota/                   # RotaRequestDTO, RotaResponseDTO  
│   ├── viagem/                 # ViagemRequestDTO, ViagemResponseDTO  
│   ├── notificacao/            # NotificacaoResponseDTO  
│   └── usuario/                # UsuarioRequestDTO, LoginRequestDTO...  
├── exception/  
│   ├── ResourceNotFoundException.java  
│   ├── BusinessException.java  
│   └── GlobalExceptionHandler.java  
├── model/  
│   └── entity/  
│       ├── veiculo/  
│       │   ├── Veiculo.java     # Entidade abstrata base  
│       │   ├── Caminhao.java  
│       │   ├── Furgao.java  
│       │   ├── Reboque.java  
│       │   └── StatusVeiculo.java  
│       ├── Motorista.java  
│       ├── Rota.java  
│       ├── Viagem.java  
│       ├── Notificacao.java  
│       ├── Usuario.java  
│       └── StatusMotorista.java  
├── pattern/  
│   ├── factory/  
│   │   └── VeiculoFactory.java   # GoF: Factory  
│   ├── adapter/  
│   │   ├── GpsService.java      # Interface Adapter  
│   │   ├── LeafletGpsAdapter.java # Provedor Leaflet/ORS  
│   │   └── CartoGpsAdapter.java  # Provedor CARTO  
│   ├── observer/  
│   │   ├── ViagemObserver.java  # Interface Observer  
│   │   ├── NotificacaoObserver.java  
│   │   └── ViagemEventPublisher.java  
│   ├── facade/  
│   │   └── ViagemFacade.java     # GoF: Facade  
│   └── strategy/  
│       ├── CustoStrategy.java    # Interface Strategy  
│       ├── CustoPorDistancia.java  
│       └── CustoPorPeso.java  
├── repository/                 # Spring Data JPA Repositories  
├── service/                    # Lógica de negócio  
└── TrabFinalApplication.java
```

- controller: Recebe requisições HTTP e delega ao Service;
- service: Regras de negócios e orquestração entre camadas;
- repository: Acesso a dados via Spring Data JPA;
- model.entity: Entidades JPA mapeadas para o banco de dados;
- dto: Objetos de transferência de dados;
- pattern.factory: Padrão Factory - criação de veículos utilizando polimorfismo;
- pattern.facade: Padrão Facade - lógica de viagem;
- pattern.strategy: Padrão Strategy - cálculo de custo por distância ou peso;
- pattern.observer: Padrão Observer - publicação de eventos de viagem.
- pattern.adapter: Padrão Adapter - integração com API de GPS;
- exception: Exceções customizadas criadas para tratar erros;
- security: Configuração JWT e acesso de usuários com ou sem permissões;

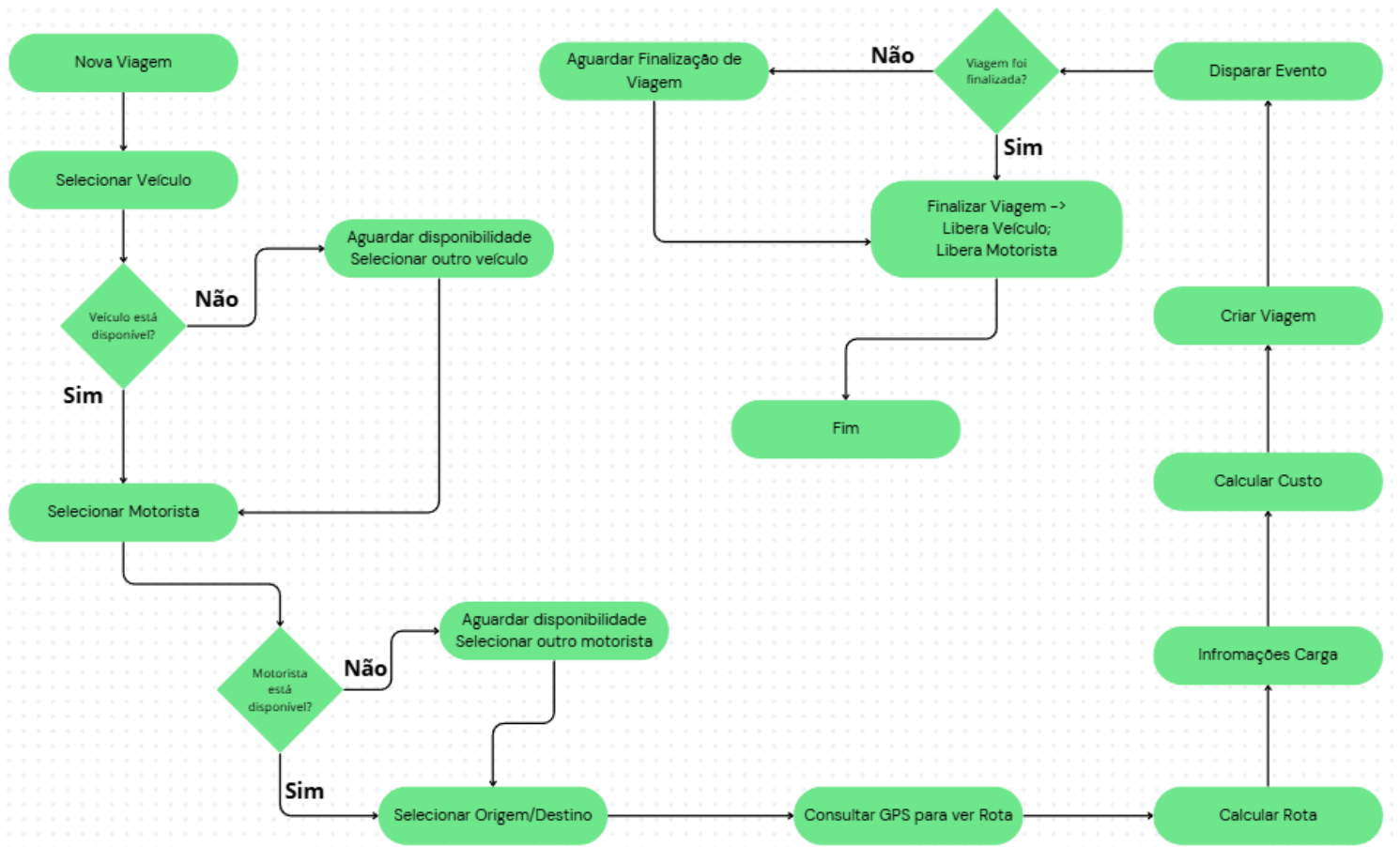
2.2 Relacionamentos Entre Entidades

Entidade	Descrição	Relacionamento
Motorista	Dados do motorista e categoria da CNH	1:N com Viagem
Veiculo	Hierarquia: Caminhao, Furgao, Reboque	1:N com Viagem
Rota	Origem, destino, distância e tempo estimado	1:N com Viagem
Viagem	Associa motorista, veículo e rota	N:1 com todos

2.3 Diagrama de Classes



2.4 Fluxograma Operacional do Projeto



3. Padrões de Projeto

3.1 Cadastro de Veículos

- Placa e chassi devem ser únicos no sistema - Validação na camada de Service antes da persistência;
- Veículos é uma classe abstrata e possui três tipos suportados: Caminhão, Furgão e Reboque, cada um possui atributos específicos;
- Todo veículo possui 3 status possíveis: *DISPONÍVEL*; *EM_VIAGEM*; *EM_MANUTENCAO*; *INATIVO*.
- Um veículo que está em viagem, em manutenção ou inativo, não pode ser utilizado em uma viagem;

3.2 Cadastro de Motoristas

- CPF e número de CNH são únicos no sistema - Validação na camada de Service antes da persistência;

- A categoria da CNH é armazenada como tipo String. Esta, limita qual veículo cada motorista pode ser associado. Por exemplo: Motorista de categoria A, B ou C não pode dirigir veículos com capacidade de KG superior a 3500 Kg, uma vez que estes veículos já são de categoria D ou E;
- Motorista possui 3 status possíveis: *DISPONÍVEL*; *EM_VIAGEM*; *AFASTADO*; *INATIVO*.
- Um motorista afastado, inativo ou em viagem não pode ser utilizado em uma viagem.

3.3 Validação de Categoria da CNH x Capacidade do Veículo

Esta regra está de acordo com o Código de Trânsito Brasileiro (CTB): Motoristas com categoria A ou B – ou qualquer combinação que não contenha C, D ou E – não estão habilitados a conduzir veículos com capacidade superior a 3.500 Kg.

- Esta verificação acontece no método *validarCategoriaCnh()* inserido em *ViagemFacade*;
- A categoria é padronizada em minúsculas antes da verificação, evitando inconsistências de cadastro dentro do banco de dados – uma vez que se trata de um String.
- Qualquer tentativa de violação desta regra ao vincular um motorista a um veículo dentro do viagem, uma mensagem é exibida ao usuário dentro de *viagem.html*

3.4 Início de Viagem

A cada viagem iniciada, as validações abaixo são executadas sequencialmente na classe *ViagemFacade* (responsável por acoplar toda a complexidade de lógica de negócio em uma classe só, retornando apenas dois métodos para o service).

#	Regra	Exceção lançada
1	Veículo deve existir	ResourceNotFoundException
2	Motorista deve existir	ResourceNotFoundException
3	Rota deve existir	ResourceNotFoundException
4	Veículo com status DISPONIVEL	BusinessException
5	Motorista com status DISPONIVEL	BusinessException
6	Peso da carga <= capacidade do veículo	BusinessException
7	Categoria da CNH compatível com o veículo (ver 3.4)	BusinessException

3.5 Finalização da Viagem

- Ao finalizar uma viagem, tanto o veículo como o motorista deve retornar automaticamente ao status de DISPONÍVEL;
- Um evento é disparado via Observer notificando a conclusão da viagem.

3.6 Cálculo de Custo

O custo de cada viagem é calculado por dois algoritmos dentro do padrão Strategy:

- POR_DISTANCIA: Custo proporcional à distância da rota em quilômetros, utilizando um multiplicador 4.5;
- POR_PESO: Custo calculado proporcionalmente ao peso da carga – valor definido ao iniciar uma viagem – e também utilizando a quilometragem total.

3.7 Rotas e Integração GPS

- Rotas devem ser cadastradas manualmente (o cálculo de distância e tempo estimado é feito pela API);
- O frontend possui a seleção de cidades através de autocomplete, ou através da seleção de cidade e estado, evitando ambiguidade ou erro na seleção;

4. Padrões de Projeto Utilizados

4.1 Factory

Padrão Factory centraliza a criação de objetos de hierarquia de veículos (Caminhão, Furgão ou Reboque), evitando que o código precise conhecer os tipos concretos. A factory recebe o DTO de requisição e retorna a instância correta de veículos conforme o campo tipo informato. Este padrão é encontrado na camada do model de veículos mais especificamente dentro da classe VeiculoFactory.

- Benefício: Facilita a adição de novos tipos de veículos sem alterar o Service ou o Controller

4.2 Facade

O Facade ficou responsável por simplificar a interface da operação mais complexa do sistema – iniciar e finalizar viagens. Eternamente, ela coordena e busca de entidades nos repositórios, executa todas as validações de negócio (disponibilidade, peso, categoria da CNH), aplica o Strategy de custo, persiste a viagem e dispara o evento Observer, expondo ao ViagemService apenas dois métodos: iniciarViagem() e finalizarViagem().

- Benefício: Reduz o acoplamento entre ViagemService e as demais camadas, concentra todas as regras de viagem em um único ponto.

4.3 Strategy

Responsável por definir uma família de algoritmos de cálculo intercambiáveis em tempo de execução. Cada implementação concreta (Custo por Distância ou Custo por Peso) implementa a classe interface CustoStrategy. A ViagemFacade seleciona a estratégia pelo nome recebido na requisição.

- Benefício: Permite adicionar novos critérios de custo sem modificar o código existente.

4.4 Observer

Ao criar ou finalizar uma viagem, a ViagemFacade fica responsável por publicar eventos através do ViagemEventPublisher. Componentes interessados (listeners) reagem a esse evento de forma desacoplada sem que a Facade precise conhecê-los.

- Benefício: Desacopla a lógica de efeitos colaterais do fluxo principal da viagem.

4.5 Adapter

A interface GpsService define o contrato de integração GPS (calcularDistancia, calcularTempoEstimado, obterRotas). Dois adaptadores implementam esse contrato: LeafLetGpsAdapter, que usa o OpenRouteService para rotas reais, e CartoGpsAdapter. A aplicação pode trocar de provedor GPS sem impacto nos serviços consumidos.

- Benefício: Isola as dependências de APIs externas; a troca de provedor GPS não exige alteração em nenhuma outra camada

5. FrontEnd

A interface é composta por páginas HTML estáticas com Bootstrap na versão 5.3 organizadas por módulos:

Página	Funcionalidade principal
dashboard.html	Visão geral: métricas, mapa com viagens ativas, relatórios
veiculos.html	CRUD de veículos com paginação e filtro por status
motoristas.html	CRUD de motoristas com paginação
rotas.html	CRUD de rotas + criação via GPS com mapa Leaflet e autocomplete
viagens.html	Listagem, filtragem, criação e finalização de viagens

6. Relatórios

O sistema faz a geração de relatórios sob demanda a partir de respectivas bibliotecas:

- PDF: Gerado através de iTextPDF, disponível para Viagens, Veículos e Motoristas;
- Excel (formato XLSX): Gerado com Apache para Viagens, Veículos e Motoristas;

7. Divisão de Responsabilidades

Integrante	Entregas	Padrões GoF
João	VeiculoFactory, LeafletGpsAdapter, CartoGpsAdapter, ViagemEventPublisher	Factory, Adapter, Observer
Luiz	Entidades JPA (Motorista, Veiculo, Viagem, Rota), Repositories, arquitetura de pacotes	—
Maiki	Todas as páginas HTML/JS, Controllers REST, geração de relatórios PDF e Excel	—
Vinicius	ViagemFacade, CustoStrategy e implementações, configuração Spring Security + JWT	Facade, Strategy

8. Banco de Dados

As tabelas do banco de dados são geradas automaticamente pelo Hibernate, refletindo o modelo de entidades da aplicação:

```
usuarios      -- autenticação e autorização
motoristas    -- cadastro de motoristas
veiculos      -- tabela base (herança JOINED)
caminhoes     -- subtipo de veículo
furgoes       -- subtipo de veículo
reboques      -- subtipo de veículo
rotas         -- origens e destinos
viagens       -- registro de viagens
notificacoes  -- sistema de notificações
```